

Documentación de seguridad de la API REST  
GA7-220501096-AA5-EV03

Nombre

Hector Wanderley Palencia Puentes

Instructor

Oscar Fernando Carvajal Castaño

Centro de Teleinformática y Producción Industrial  
Análisis y Desarrollo de Software

Ficha: 3186640

2026

Este documento describe los mecanismos de seguridad implementados en la API REST de SIAPE para proteger los endpoints, los datos y las operaciones del sistema.

## Autenticación

La API utiliza autenticación en dos pasos:

### Paso 1 — Verificación de credenciales:

El usuario envía su `user_name` y `password`. El sistema valida las credenciales comparando el `password` con el hash almacenado en la BD usando `bcrypt`. Si son correctas, genera un código numérico de 6 dígitos y lo envía al correo registrado del usuario.

### Paso 2 — Verificación del código:

El usuario envía el código recibido por correo. Si es válido y no ha expirado, el sistema genera y retorna un token JWT firmado con `JWT_SECRET` y con expiración de 8 horas.

## Autorización con JWT

Todos los endpoints excepto `/api/health`, `/api/auth/login` y `/api/auth/verify-code` requieren un token JWT válido en el header de la solicitud:

Authorization: Bearer <token>

El middleware `verifyToken` valida el token en cada solicitud antes de llegar al controlador. Si el token es inválido o ha expirado, retorna HTTP 403.

## Roles de usuario

El sistema maneja dos roles definidos en el campo `cargo` del token JWT:

| Rol        | Descripción                                       |
|------------|---------------------------------------------------|
| admin      | Acceso completo a todos los módulos y operaciones |
| otro cargo | Acceso de solo lectura a los módulos permitidos   |

Las rutas de administración como `/api/trabajadores` y `/api/reportes` solo son accesibles para usuarios con cargo `admin`.

## Protección de contraseñas

- Las contraseñas se hashean con **bcrypt** antes de guardarse en la BD
- El salt se regenera en cada cambio de contraseña
- Los campos `password_hash` y `salt` nunca se retornan en ninguna respuesta de la API
- Un administrador no puede cambiar la contraseña de otro administrador

### **Códigos de verificación**

- Los códigos son numéricos de 6 dígitos generados aleatoriamente
- Se almacenan en memoria (no en BD) con expiración de 10 minutos
- Son de un solo uso — se eliminan del Map al ser verificados correctamente
- El mismo mensaje de error se retorna para código incorrecto y código expirado, evitando enumeración

### **Protección de datos sensibles**

- Las credenciales del servidor de correo se almacenan en variables de entorno (.env) nunca expuestas en el código
- El archivo .env está excluido del repositorio en .gitignore
- El remitente de los PQRS se toma del token JWT, no del body de la solicitud, evitando suplantación
- Los errores del backend no exponen detalles internos de MySQL al cliente

### **Integridad de datos**

- Las operaciones críticas (creación de pedidos, facturas, eliminación de inventario) se ejecutan dentro de transacciones SQL con rollback automático ante fallos
- Las claves foráneas en la BD previenen eliminación de registros con dependencias activas
- Los campos permitidos de actualización están definidos en listas blancas en cada servicio, evitando que el cliente modifique campos no autorizados como password\_hash o salt

### **Expiración de sesión**

- El token JWT expira automáticamente a las 8 horas de su emisión
- El frontend detecta la expiración y redirige automáticamente al login
- El interceptor de axios maneja los errores 401 y 403 limpiando el localStorage y redirigiendo al login